Newton School of Technology

13

Lecture 13

Process Synchronization IV: Semaphores

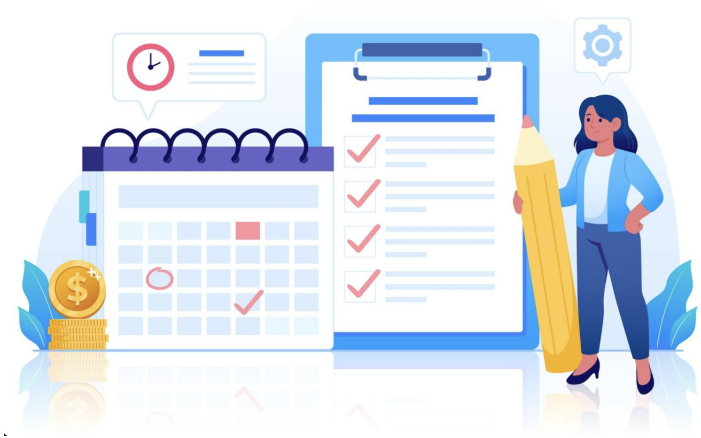
by - Brijesh Shukla

C07: Operating Systems

Today's Agenda

Newton School of Technology

- Define Semaphores and their historical context (Dijkstra).
- Differentiate Binary vs. Counting Semaphores.
- Master the atomic operations: `wait()` (P) and `signal()` (V).
- Analyze OS implementation: Queues, Atomicity, and Blocking.
- Identify common pitfalls: Deadlock, Starvation, and Priority Inversion.



Recap: The Synchronization Hierarchy

Newton School of Technology

Hardware (TAS/CAS):

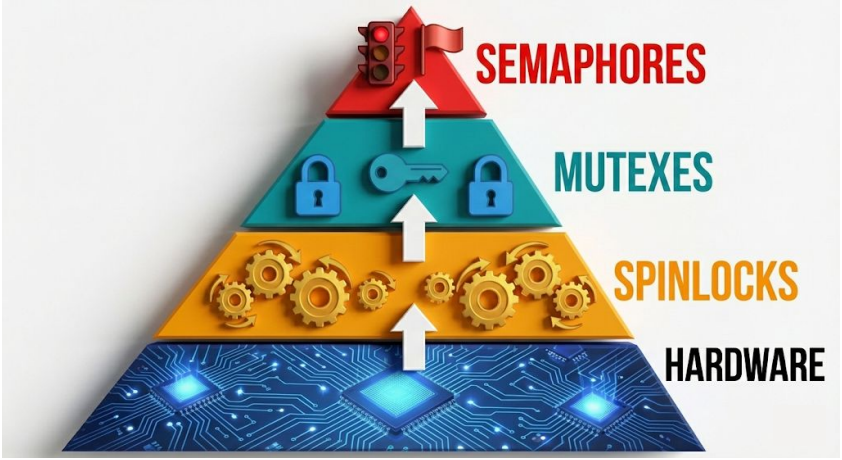
- Fast but complex; difficult to program directly.

Spinlocks:

- Busy-waiting wastes CPU cycles.
- Useful only for very short critical sections.

Mutexes:

- Simple locks (0 or 1).
- Strict ownership: Only the locker can unlock.



The Problem with Mutexes

Newton School of Technology

Scenario: You have 5 Database Connections and 50 Threads.

The Limitation:
A Mutex is binary: It is either locked or unlocked.

It cannot count **"5 available slots."**

If you have **3 printers and 10 threads,** how do you coordinate access using only mutexes?

Answer: It requires messy counters and multiple locks. We need a better abstraction.

Enter the Semaphore

Newton School of Technology

Definition: A synchronization primitive introduced by **Edsger Dijkstra (1965).**

Concept: An integer variable accessed only through two atomic operations.

Versatility:

- Mutual Exclusion (Locking).
- Resource Counting (Pools).
- Event Signaling (Synchronization).

Name Origin: Railroad signals (P = Proberen/Test, V = Verhogen/Increment).

The Semaphore Abstract Data Type

Newton School of Technology

It is not just an integer.

```
typedef struct {
    int value;              // Counter
(available resources)
    queue waiting_list;  // List of
blocked threads
} semaphore;
```

Binary Semaphore

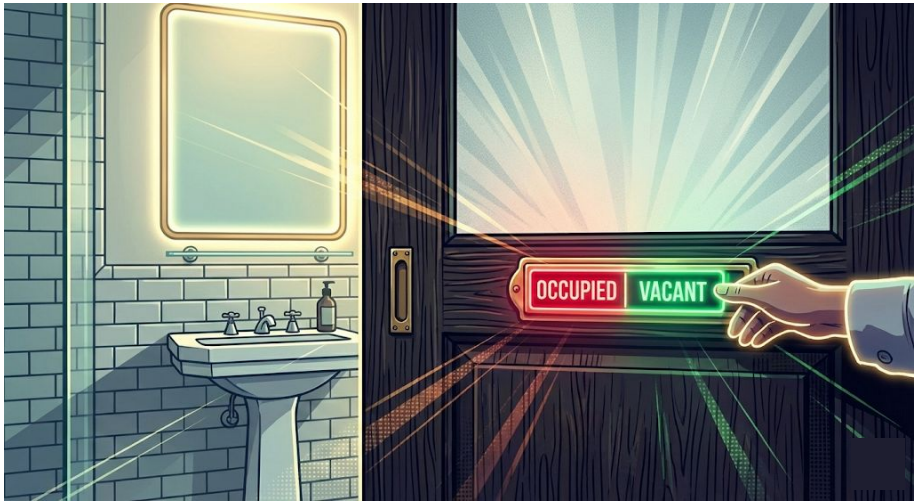
Newton School of Technology

Value Range: 0 or 1.

Behavior: Acts like a Mutex.

- 1: Available (Unlocked).
- 0: Locked (Wait).

Use Case: Protecting a Critical Section.



Counting Semaphore

Newton School of Technology

Value Range: 0 to N.

Initial Value: Total resources (e.g., `N=5` connections).

Logic:

- `Value > 0:` Resources available.
- `Value = 0:` All taken.
- `Value < 0:` Threads are waiting.

Visualizing the Difference

Newton School of Technology

Binary:
`[1]` → `[0]` (Lock) → `[1]` (Unlock)

Counting (Init 3):

`[3]` → `[2]` → `[1]` → `[0]` (Full) → Wait

How is a counting semaphore (`N=3`) different from 3 binary semaphores?

Answer: The counting semaphore manages the pool as a single unified group.

The Two Atomic Operations

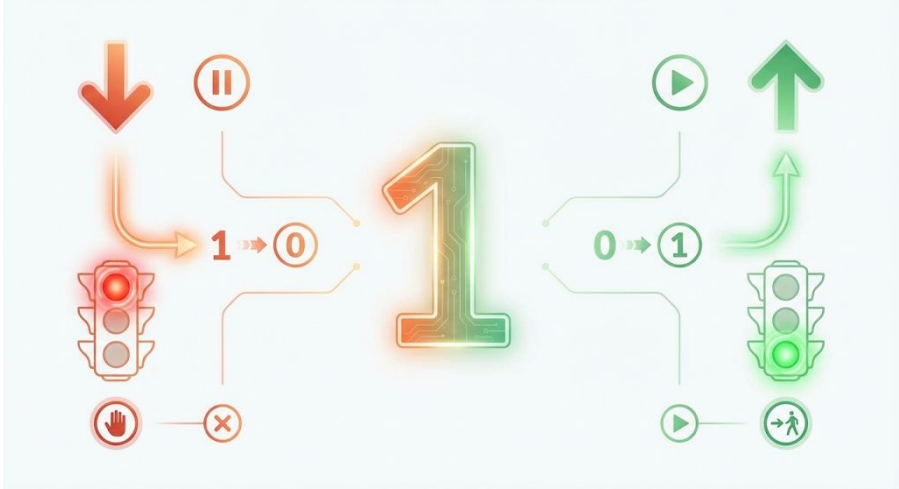
Newton School of Technology

Wait() (P, down, acquire):

- Decrements value.
- Blocks if resources are unavailable.

Signal() (V, up, release):

- Increments value.
- Wakes a waiting thread (if any).
- Never blocks.




```
wait(S) {
    S→value--;
    if (S→value < 0) {
        add_to_queue(current_thread);
        block(); // Puts thread to sleep!
    }
}
```

Crucial Insight: This yields the CPU. No busy waiting.

Signal : Pseudo Code

Newton School of Technology

```
signal(S) {
    S→value++;
    if (S→value ⩽ 0) {
        remove_thread_from_queue();
        wakeup(thread);
    }
}
```

The Rule of Magnitude:

If value is negative:

- (e.g., -3), the magnitude (3) equals the number of sleeping threads.

If value is zero:

- 0 resources left, but 0 threads waiting. (The boundary case).

Crucial Insight: If value was negative, it means someone was waiting.

Simulation Trace

Newton School of Technology

Scenario: Init = 3 (Printers). 5 Threads request access.

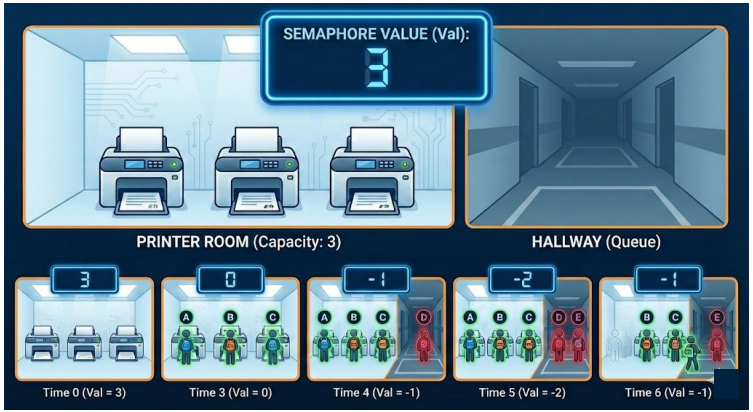
Trace:
Time 0: Val = 3.

Time 3 (A,B,C wait): Val = 0. (All taken).

Time 4 (D waits): Val = -1. Queue: [D].

Time 5 (E waits): Val = -2. Queue: [D, E].

Time 6 (A signals): Val = -1. D wakes up.



Busy Waiting (The Spinlock Approach)

Newton School of Technology

Mechanism: Thread loops constantly (`while(!free){}`), staying in the Running state.

CPU: High impact. Core locked at 100% usage doing "useless" checks.

Latency: Ultra-low. Immediate access when available (nanoseconds).

Use Case: Tiny critical sections (< 1µs).

Blocking (Semaphore)

Newton School of Technology

Mechanism: Thread yields CPU (`block()`), moving to the Waiting state.

CPU: Zero usage. OS schedules other work.

Latency: Moderate. Requires expensive context switch to wake up (microseconds).

Use Case: Long waits (I/O, DB) or high contention.

Inside the OS Kernel: The Data Structure

Newton School of Technology

The "Black Box" Revealed: A semaphore isn't just a number; it's a kernel object.

Kernel Struct Components:
Atomic Integer (`count`): Tracks resources. Must be manipulated atomically.

Wait Queue (`wait_list`): A linked list of Thread Control Blocks (TCBs) representing sleeping threads.

Lock (`spinlock`): A tiny internal lock to protect the integrity of the struct itself during updates.

Why Kernel Space?
Because "**blocking**" a thread requires **changing its state** in the **Scheduler**, which is **a privileged OS operation.**

The Atomicity Challenge

Newton School of Technology

The Critical Race: If Thread A calls `wait()` and Thread B calls `signal()` at the exact same nanosecond on different cores, they might both read the same `count` value before updating it.

Result: Corrupted state (e.g., `count` is -1 but no one is in the queue).

The Fix: We need a mechanism to make the entire `if (count < 0) { ... }` block indivisible.

The "Spinlock Inside" (Implementation Irony)

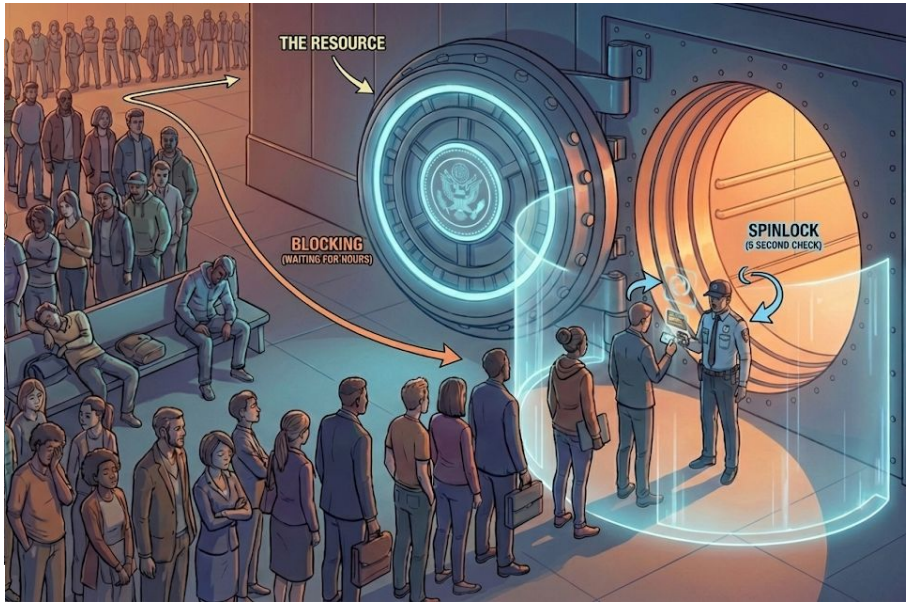
Newton School of Technology

The Paradox: "I thought Semaphores were used to avoid spinlocks?"

The Reality: We use a Spinlock inside the Semaphore implementation, but its scope is tiny.

- Scope: It only protects the 10-20 assembly instructions needed to increment `count` and add a node to the linked list.
- Duration: Nanoseconds.

Key Distinction: Threads spin to update the metadata, but they block (sleep) to wait for the resource.



Modern Optimization: The "Two-Phase" Wait

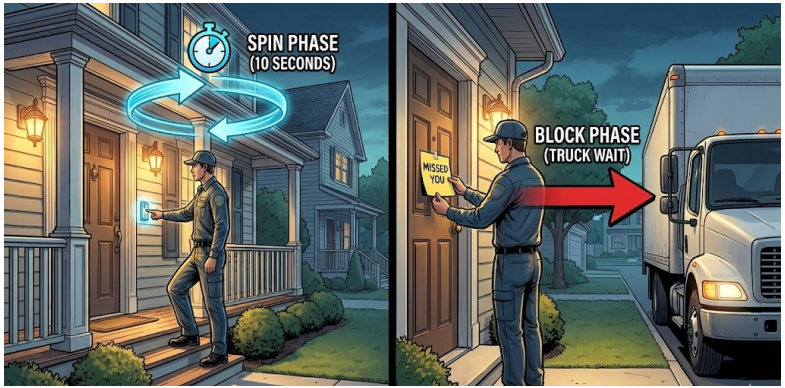
Newton School of Technology

The Problem: Putting a thread to sleep (Context Switch) is "expensive" (~5-10 microseconds). If the resource frees up in 1 microsecond, we wasted time sleeping.

The Solution **(Linux Futex / Windows Dispatcher):**

- **Phase 1 (Spin):** Spin for a tiny limit (e.g., 50 iteration). Is it free yet? Is it free yet?
- **Phase 2 (Block):** If still unavailable, then yield the CPU and sleep.

Benefit: Gives the responsiveness of a spinlock for short waits and the efficiency of a semaphore for long waits.



Queue Policies & Fairness

Newton School of Technology

Strict FIFO (First-In-First-Out):

- The thread that has waited longest wakes up first.
- Pro: Strong Semaphore. Guarantees no starvation.

Priority-Based:

- The thread with the highest OS priority (e.g., Real-Time Audio) wakes up first, jumping the line.
- Con: Weak Semaphore. Low-priority threads might never wake up if high-priority threads keep arriving.

The Standard API (POSIX)

Newton School of Technology

Library: `<semaphore.h>` (Standard on Linux/Unix).

`sem_t s;` //Initialize

The "Big 4" Functions:
`sem_init(&s, pshared, value):` Set initial count. pshared=0 for threads.

`sem_wait(&s):` The blocking "Down" operation.

`sem_post(&s):` The signaling "Up" operation.

`sem_destroy(&s):` Clean up kernel resources.

Return Values: Always check for -1! (Interrupted system calls are common).

The Binary Mutex Pattern

Newton School of Technology

Objective: Protect a shared counter from race conditions.

Implementation:

```
sem_t mutex;
int counter = 0;
sem_init(&mutex, 0, 1)
```

(Important: 1 means "unlocked").

```
sem_wait(&mutex);   // Lock
counter++;          // Critical Section
sem_post(&mutex);   // Unlock
```

What happens if you initialize to 0?

```
sem_init(&mutex, 0, 0);

sem_wait(&mutex);   // Blocks immediately!
// Deadlock: Thread waits forever for itself to release
```

Always initialize binary semaphore to 1 for mutual exclusion!

Pattern: The Resource Pool

Newton School of Technology

Scenario: Managing `N` Database Connections.

Logic:

- Init: `sem_init(&pool, 0, N).`
- Acquire: `sem_wait(&pool)` returns immediately for the first N threads.
- Block: The (N+1)th thread blocks until someone calls `sem_post`.

Efficiency: No "manager thread" needed. The kernel handles the queuing automatically.

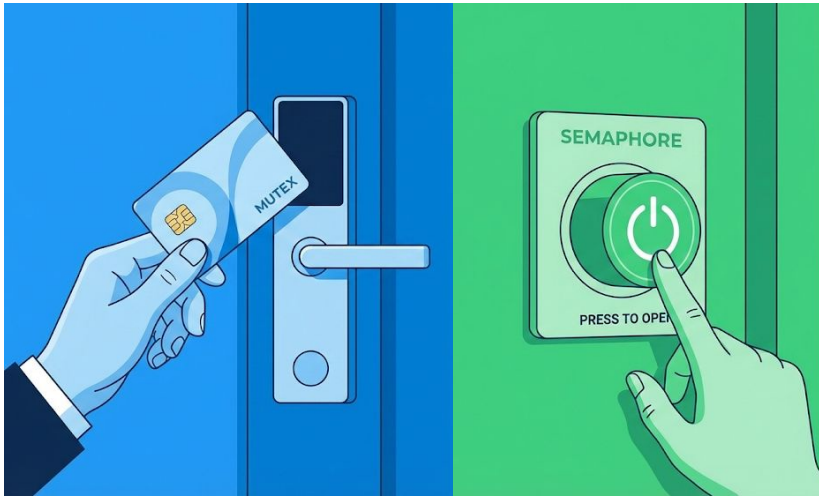
Deep Dive: Mutex vs. Semaphore

Newton School of Technology

Mutex: "I locked it, so only I can unlock it." (Good for sanity/debugging).

Semaphore: "Someone needs to wait, someone needs to signal." (No ownership).

Consequence: A thread can `post()` a semaphore it never `wait()`ed on. This is legal (and necessary for Signaling) but impossible with a Mutex.



Mutex: A personal hotel key card (User specific).

Semaphore: A generic "Press to Open" door button (Anyone can press it).

Pitfall 1: Deadlock (The Deadly Embrace)

Newton School of Technology

Definition: A circular chain of waiting threads. T1 waits for T2, T2 waits for T1.

The Code of Doom:

- **Thread 1:** `wait(A); wait(B);`
- **Thread 2:** `wait(B); wait(A);`

The "Context Switch" Trigger: T1 gets A, T2 gets B. Both block. Game over.

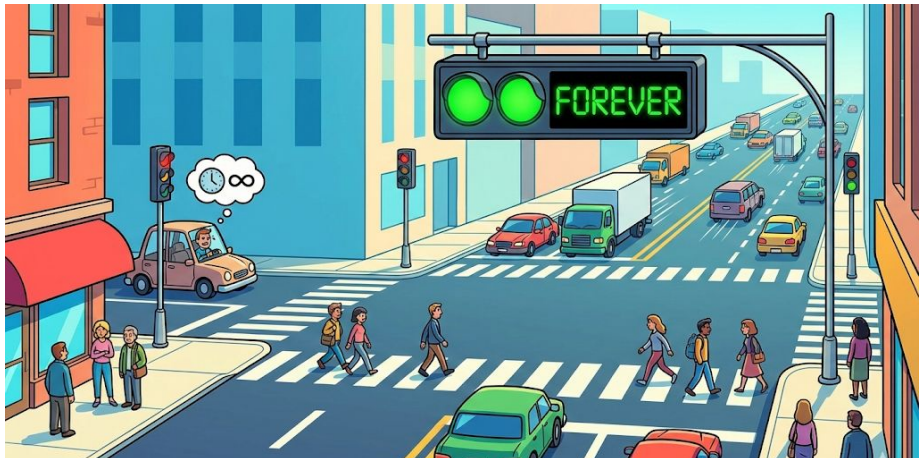
Pitfall 2: Starvation & Fairness

Newton School of Technology

Definition: The system is making progress, but one specific thread is stuck waiting forever.

Cause: "Weak" Semaphore implementation or aggressive High-Priority threads.

The "Polite" Problem: If the semaphore queue is LIFO (Last-In-First-Out), a steady stream of new threads could keep an old thread buried at the bottom of the stack.



A busy intersection where the traffic light stays green for the main road forever, and the car on the small side street never gets a turn.

Common Coding Bugs (Logic Errors)

Newton School of Technology

Bug 1: Unbalanced Signal: Calling `post()` one too many times.

Result: `count` exceeds `N`. Two threads enter a critical section designed for one. (Safety Violation).

Bug 2: The Lost Wakeup: Forgetting to `post()` on an error path (e.g., inside an `if (error) return;` block).

Result: The resource leaks. Eventually, all threads block.

A "Leak" in a pipe. The water (resources) drips out until the pipe is dry (System halts).

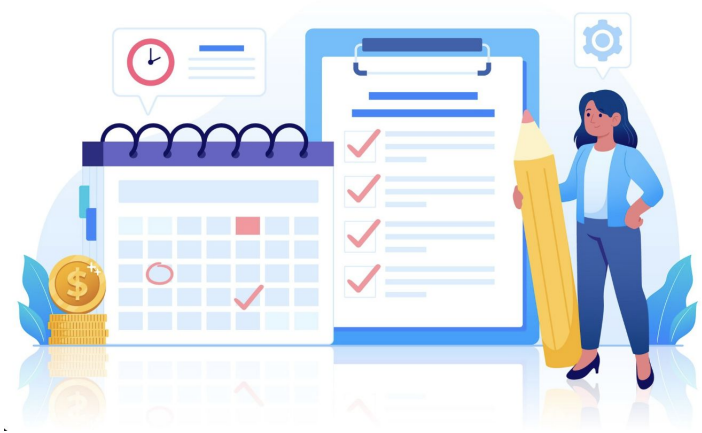
Best Practices Checklist

Newton School of Technology

Lock Ordering: Always acquire Mutex A before Mutex B.

RAII (Resource Acquisition Is Initialization): Use C++ destructors or `defer` (in Go/Swift) to ensure `signal()` is always called, even if code crashes.

Keep Critical Sections Short: Don't do I/O or network calls inside a binary semaphore lock.



Thank You!

Newton School of Technology

Thank you for your attention. Feel free to ask questions or revisit the topics covered.